
Early-Exit Graph Neural Networks for Link Prediction

Roman Knyazhitskiy
University of Cambridge
rk804@cam.ac.uk

Andrea Giuseppe Di Francesco
Sapienza University of Rome

Abstract

Graph Neural Networks are great for link prediction in various network-like structures; however, the question of their speed/quality tradeoff has been barely studied. While in practice the time it takes to do inference matters little for small benchmarks, the latency does limit applicability in large-scale domains. In this work, we explore early-exiting strategies that can be applied to Graph Neural Networks to solve the problem of link-prediction faster. We are largely inspired by the method described in [1]: we use no auxiliary losses to enforce early exiting, allowing it to emerge as an implicit property of the architecture. We show that our method enables early exiting in several setups, moving the Pareto frontier on the HeaRT benchmark for GCN and SAS-GNN backbones. Our findings show that inference speed of GNNs on many link-prediction problems can be improved, while losing little, or even winning in terms of prediction quality. The code is available in our repository: https://github.com/knyazer/link_prediction.

1 Introduction

Graph Neural Networks (GNNs) are the standard tool for learning on graph-structured data. Protein folding Jumper et al. [2], citation networks Kipf and Welling [3], and social networks Hamilton et al. [4] are domains where the inductive biases of GNNs seem to work well.

A fundamental issue for deep GNN architectures is *over-smoothing*: as depth increases, node embeddings converge to indistinguishable vectors [5, 6]. Attention-based GNNs mitigate this to some extent by being better at filtering irrelevant messages, but most architectures still saturate beyond roughly 20 layers Rusch et al. [6]. Several remedies exist, including stochastic regularization Zhang et al. [7] and architectures with provable stability guarantees such as the Symmetric-Anti-Symmetric GNN (SAS-GNN) [1, 8].

Beyond over-smoothing, GNNs are *non-adaptive*: every node undergoes the same number of message-passing steps regardless of local difficulty. Consider a graph with two highly connected subgraphs - one dense (diameter < 3) and another sparse (diameter > 5). Predicting new links in the dense subgraph is inherently easier; spending the same compute on both wastes resources on the harder, less predictable subgraph.

Early exiting [9] addresses this by detecting, via intermediate representations, whether further computation is warranted. Most early-exit methods require an auxiliary budget loss to encourage halting [10]. Francesco et al. [1] demonstrated that for GNNs built on stable Neural ODE backbones, early exiting emerges naturally without such losses: later layers can have measurably worse prediction quality than earlier ones, so the task loss alone drives the exit policy.

Their work, however, targets node and graph classification. *Link prediction* introduces additional challenges. First, predictions depend on *pairs* of node embeddings, so freezing one node’s representation affects all candidate edges involving that node. Second, the relevant subgraph structure differs across candidate edges, making per-edge adaptive compute especially appealing. To our knowledge, no prior work has applied auxiliary-loss-free early exiting to link prediction.

Our contributions are two-fold:

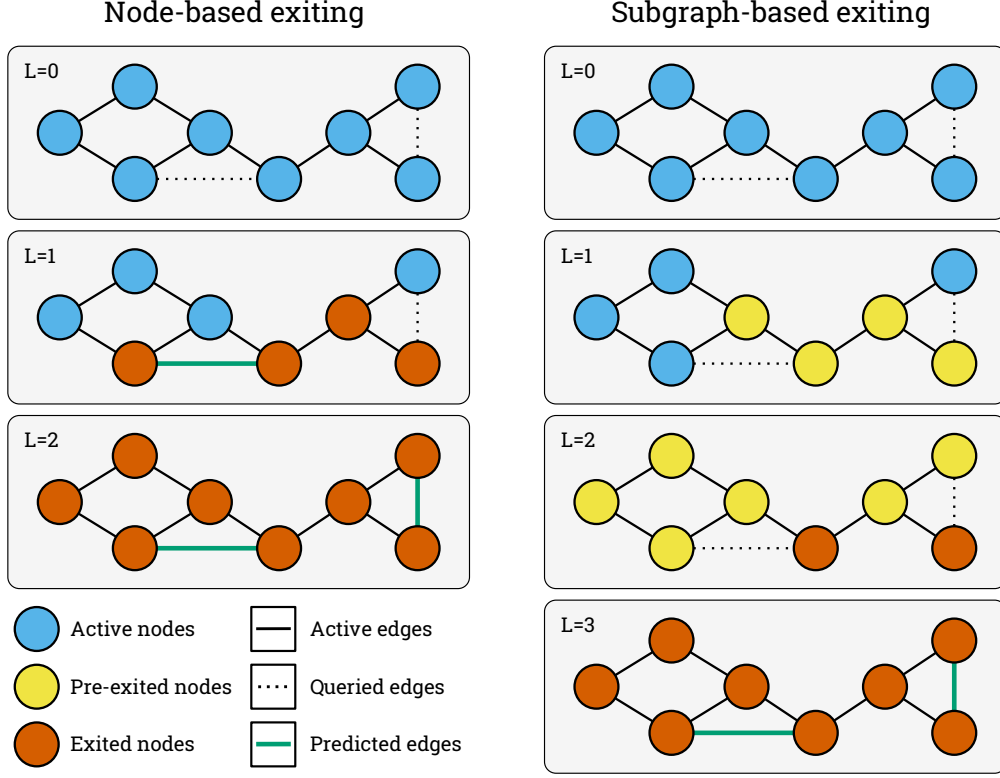


Figure 1: A visualization of both of our methods. *Left:* node-based exiting, where the link is predicted when both of the nodes it depended on are exited. *Right:* subgraph-based exiting, where the link is predicted when both of the nodes it depended on exited, but the neighbourhood around each node is also marked as "ready to exit".

1. We propose **node-level** and **subgraph-level** early-exit strategies for link prediction in GNNs. Both are trained end-to-end with only the task loss, following the EEGNN framework [1], and we adapt them to operate on the HeaRT benchmark [11].
2. We demonstrate empirically that early exiting shifts the performance-compute Pareto frontier on most HeaRT datasets for both GCN and SAS-GNN backbones, providing higher MRR and Hit@ metrics for the smaller compute budget.

2 Method

Notation. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathbf{X})$ be an undirected graph with node set $\mathcal{V}$, edge set $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$, and node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$, where $n = |\mathcal{V}|$. The adjacency matrix is $\mathbf{A} \in \{0, 1\}^{n \times n}$ and the normalised adjacency matrix is $\bar{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, where $\mathbf{D}$ is the degree matrix. At layer l , the hidden node embedding matrix is $\mathbf{H}^l \in \mathbb{R}^{n \times m}$, with $\mathbf{H}^0 = f_{\text{enc}}(\mathbf{X})$ for a learnable encoder f_{enc} . For a node u , we write $\mathbf{h}_u^l \in \mathbb{R}^m$ for its embedding at layer l .

Link prediction setting. Given a candidate edge (u, v) , we predict its existence from node embeddings $\mathbf{h}_u^l$ and $\mathbf{h}_v^l$ via a decoder $g : E \times E \rightarrow \mathbb{R}^2$.

Backbone. We experiment with two backbones: a standard GCN [3], with small tweaks from the HeaRT codebase, and SAS-GNN [1]. The SAS-GNN update rule is:

$$\mathbf{H}^{l+1} = \mathbf{H}^l + \tau \sigma_1(-\sigma_2(\mathbf{H}^l \Omega_{\text{as}}) + \bar{\mathbf{A}} \mathbf{H}^l \mathbf{W}_s), \quad (1)$$

where $\Omega_{\text{as}} = \Omega - \Omega^\top$ is an antisymmetric weight matrix, $\mathbf{W}_s = \mathbf{W}_s^\top$ is a symmetric weight matrix, τ is the integration step, and $\sigma_1 = \text{ReLU} \circ \tanh$, $\sigma_2 = \text{ReLU}$. This ODE-inspired design is provably

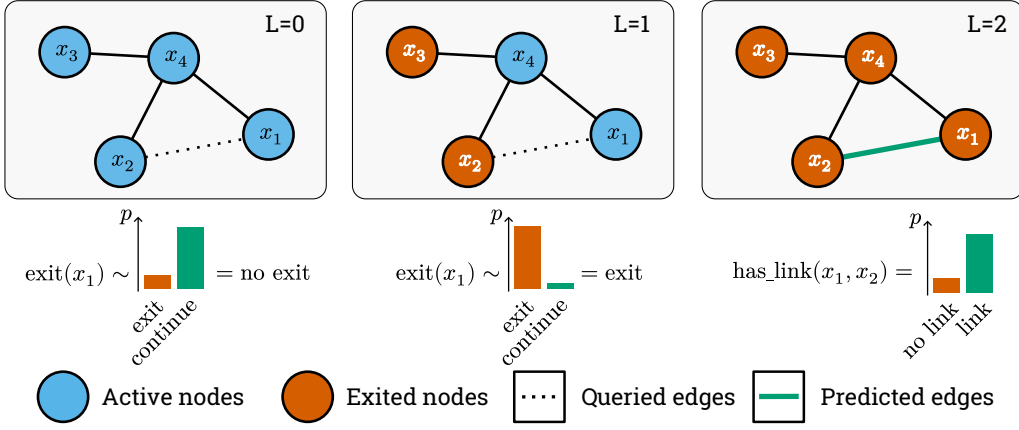


Figure 2: Node based early exiting visualization. On each layer, each node decides whether it wants to exit - which implies that the embeddings of the node will be frozen. It does so by doing a relaxation between greedy and categorical sampling from its decision (continue/exit) output logits with gumbel-softmax trick.

stable and non-dissipative [1, 4], ensuring that intermediate representations remain informative. While originally argued to be a prerequisite for early-exiting, we find in practice that early exiting is feasible for standard GNN architectures too. We show this in Section 4.

Gumbel-Softmax exit mechanism. Following Francesco et al. [1], we attach a lightweight confidence head at each node and each layer. For node u at layer l , a shared GNN module f_c produces a confidence vector $\mathbf{C}_u^l \in \mathbb{R}^2$ over actions $\{\text{CONTINUE}, \text{EXIT}\}$. A second module f_ν predicts a temperature $\nu_u^l > 0$. The Gumbel-Softmax score is:

$$\mathbf{c}_u^l = \text{Softmax}\left(\frac{\log \mathbf{C}_u^l + \mathbf{g}_u^l}{\nu_u^l}\right), \quad \mathbf{g}_u^l \sim \text{Gumbel}(0, 1), \quad (2)$$

which approaches a one-hot vector as $\nu_u^l \rightarrow 0$. We use the straight-through estimator [12] for backpropagation. The hidden dimension of f_c and f_ν is usually chosen to be $m_f = 32$, compared to $m = 256$ for node embeddings, keeping overhead small.

The reason to choose Gumbel is purely heuristic: it seems to behave better in our setup than the standard categorical straight-through estimator.

2.1 Node-based early exiting

Each node independently decides whether to exit at each layer. The integration step in Eq. (1) becomes node- and layer-dependent:

$$\tau_u^l = c_u^l(0), \quad (3)$$

where $xc_u^l(0)$ is the CONTINUE probability. When a node exits ($c_u^l(0) \rightarrow 0$), its embedding is frozen: $\mathbf{h}_u^{l+1} = \mathbf{h}_u^l$, but the node *still participates in message passing* for its neighbours' updates. The final embedding used for link prediction is $\mathbf{z}_u = \mathbf{h}_u^{l^*}$, where $l_u^* = \min\{l : \arg \max \mathbf{c}_u^l = 1\}$, or $l^* = L$ if the node never exits.

The heads f_c and f_ν are trained with only the link prediction loss (e.g. binary cross-entropy over positive and negative edges). No auxiliary budget loss is used; early exiting emerges from the task objective alone.

2.2 Subgraph-based early exiting

Node-level exiting might lead to some inconsistencies: a node may freeze while the neighbors still update, which might create a mismatch in the information flow of adjacent embeddings; for instance,

maybe a node is aware of the need to propagate its embedding to neighbors, but neighbors are not aware of that and just exit. To address this, we introduce an exit protocol that conditions each node’s exit on its neighborhood’s readiness.

Phase 1: Soft exit. At each layer l , a shared confidence head f_s predicts a soft exit score for each node. Using the Gumbel-Softmax mechanism from Eq. (2), we obtain a binary signal $e_u^l \in \{0, 1\}$ indicating whether node u considers itself ready to exit. We track the cumulative soft-exit state $\bar{e}_u^l = 1 - \prod_{k=1}^l (1 - e_u^k)$, which increases as the node becomes more confident.

Phase 2: Hard exit via neighborhood readiness. Before a node actually freezes, we check whether its local neighborhood shares its confidence. We compute a readiness score as the mean soft-exit state of its neighbors:

$$R_u^l = \frac{1}{|\mathcal{N}(u)|} \sum_{v \in \mathcal{N}(u)} \bar{e}_v^l. \quad (4)$$

A separate hard exit head f_h , a two-layer MLP, takes the concatenation $[\mathbf{h}_u^l; R_u^l]$ as input and produces logits over $\{\text{CONTINUE}, \text{EXIT}\}$, which are again passed through Gumbel-Softmax. The effective exit probability is gated by the node’s own soft-exit state, so that a node can only hard-exit once it has marked itself as ready:

$$p_u^l = \bar{e}_u^l \cdot c_u^{l, \text{hard}}(1), \quad (5)$$

where $c_u^{l, \text{hard}}(1)$ is the hard exit probability, outputted by f_h . This forces dense subgraphs to exit together: a node waits until its neighbors are also ready before freezing its embedding.

2.3 Theoretical Compute.

We approximate the compute cost at layer l as:

$$C^{(l)} = n_{\text{active}}^{(l)} \cdot h^2 + e_{\text{active}}^{(l)} \cdot h, \quad (6)$$

where h is the hidden dimension. For the baseline (no early exiting), $n_{\text{active}}^{(l)} = N$ and $e_{\text{active}}^{(l)} = E$ at every layer. Note that these FLOP savings should correspond to wall-clock speedups on CPU; on GPU, realising them requires sparse-execution support or custom kernels, since frozen nodes still occupy tensor rows in a naive dense implementation.

2.4 Implementation Notes

We largely follow the EEGNN implementation of [1]. We identified one critical instability: the original temperature function f_ν can output values near zero, causing the softmax gradients in Eq. (2) to diverge to infinity or produce NaN values if trained for too long. We resolve this by clamping ν_u^l from below:

$$\nu_u^l = \max(f_\nu(\mathbf{H}^l), 0.1). \quad (7)$$

This simple fix stabilises training for deep networks ($L \geq 20$) and is necessary for convergence. The threshold 0.1 was not tuned; moderate variations (e.g. 0.05–0.2) did not noticeably affect performance.

3 Evaluation

3.1 Early exit depth distribution

We look at how the early exit depth is distributed for a few datasets; as in, does the exit depth distribution change meaningfully for deeper nets. We notice an interesting fact: while for deeper networks there is an emergence of the desired behaviour, it appears as a consistent bump near the intrinsic complexity of the dataset. Figure 3 shows the exit layer distribution for ResGCN on Citeseer and PubMed across depths $L \in \{4, 6, 8, 12, 16\}$. On Citeseer, the distribution is sharply concentrated at the first few layers regardless of network depth: nearly all nodes exit immediately, suggesting that the link prediction task on this graph requires very little message passing. This is consistent with Citeseer being a small, relatively dense citation network where local features are already highly informative; indeed, an MLP on node features alone can achieve competitive accuracy on this dataset. Surprisingly we can see two distinct modes: shallower networks are more reluctant to start exiting early, and have around 10% probability of exiting at layer 2, while deeper networks collapse harder.

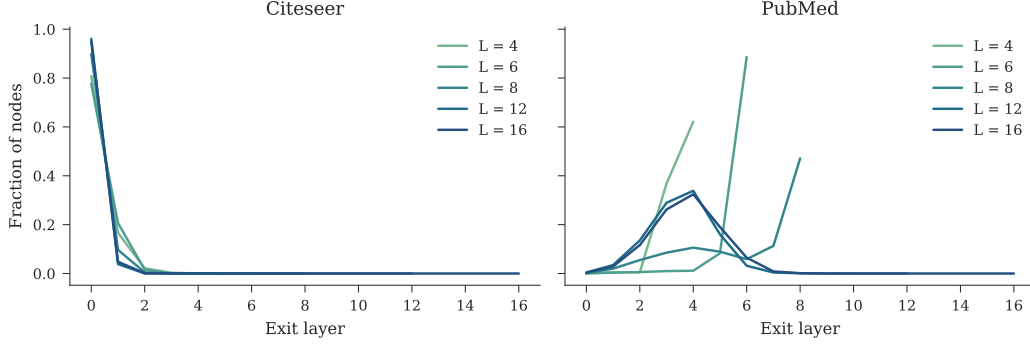


Figure 3: Early exiting depth distribution of GCN for PubMed and Citeseer. While Citeseer seems to be an easy problem and collapses to the very small depth, PubMed behaves more interestingly. While for smaller depths it learns to never exit, it shows a distinct bump around layer 4 for deep networks; most of the nodes exit around this layer, with few exiting before and after. Shown are the averages of three training runs.

3.2 Performance and Metrics

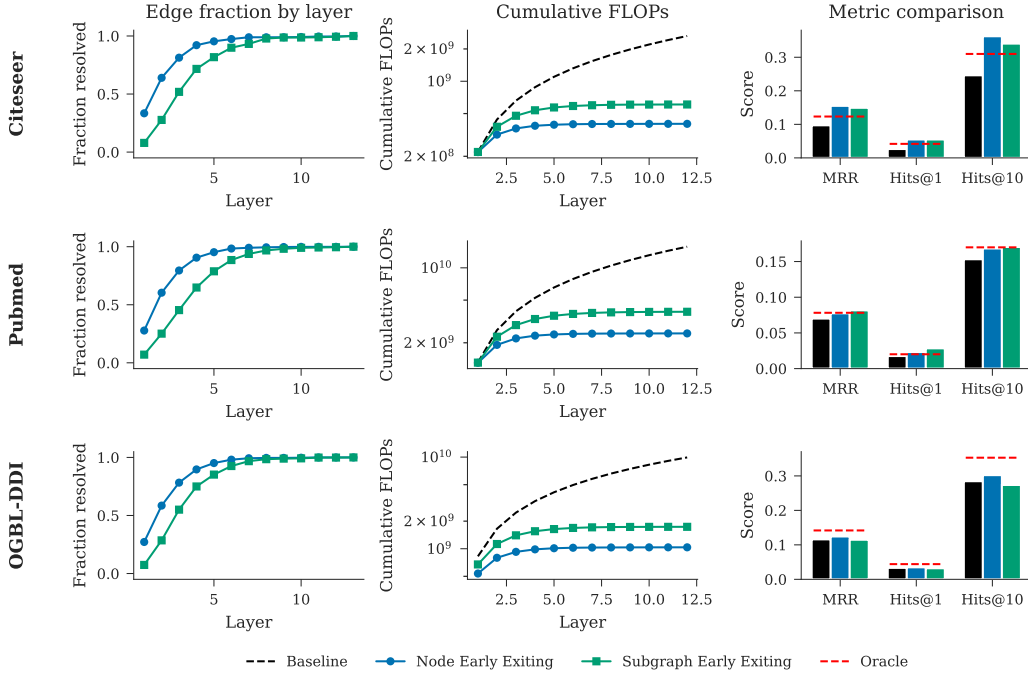


Figure 4: Evaluation of GCN via HeaRT on a few datasets. Our method outperforms the baseline, and sometimes even outperforms the oracle, which is a GCN with optimally chosen depth. Edge fraction by layer corresponds to the number of exited edges per layer, with 1 corresponding to all edges being done. Cumulative FLOPs are the amount of compute the model has spent up to a particular layer.

We evaluate node-level and subgraph-level early exiting on three HeaRT benchmark datasets: Citeseer, Pubmed, and ogbl-ddi. We compare against a fixed-depth baseline (no early exiting) and an oracle that trains separate fixed-depth GCNs for $L \in \{1, 3, 5, 7, 9, 11\}$ and selects the depth with the highest validation MRR. All experiments use a GCN backbone with $L=15$ layers unless stated otherwise.

Figure 4 summarises the results. The left column shows the fraction of nodes resolved (exited) as a function of layer. Node-level exiting resolves nodes faster than subgraph exiting on all three datasets,

which is expected: the neighbourhood consensus constraint in subgraph exiting delays individual nodes from freezing. On Citeseer and PubMed, both methods resolve nearly all nodes by layer 8–10, well before the final layer. The middle column shows cumulative FLOPs (computed via Eq. 6). Both early-exit strategies use substantially less compute than the baseline across all datasets. Node exiting is consistently cheaper than subgraph exiting, since it freezes nodes more aggressively. On Citeseer, node exiting uses roughly $5\times$ fewer FLOPs than the baseline at layer 15.

The right column shows link prediction quality (MRR, Hits@1, Hits@10). On Citeseer, subgraph exiting matches or exceeds the oracle on Hits@10, and both methods substantially outperform the baseline on all metrics. On PubMed, the picture is more mixed: node exiting underperforms the baseline on MRR and Hits@1, while subgraph exiting stays competitive. On ogbl-ddi, both methods improve over the baseline on Hits@10, though MRR results are comparable.

3.3 Hyperparameter sweep

For GCN evaluations we do 3 runs to tune the learning rate, and then 3 runs to compute the distribution of losses at a particular value, thus a total of 5 runs, with one being reused between the two stages. This is similar to a HeaRT benchmark setup, but with a lot of compute saved. We couldn’t run some datasets due to them being too large or not well-supported by HeaRT implementation, and we decreased the maximum number of allowed steps by a factor of 10 to accomodate limited time. The results can be seen in Table 1.

Table 1: MRR (%) on HeaRT benchmark (500 hard negatives). **Fixed:** standard GCN without early exiting. **NodeExit:** node-level early exiting (Section 2.1). **SubgraphExit:** subgraph-level early exiting (Section 2.2). LR tuned from {0.0005, 0.001, 0.002}, 3 seeds. Best result per row in **bold**.

Dataset	L	Fixed	NodeExit	SubgraphExit
Citeseer	1	20.39 $\pm$ 1.03	20.51 $\pm$ 1.15	20.30 $\pm$ 0.28
	2	19.86 $\pm$ 0.68	20.94 $\pm$ 0.67	20.24 $\pm$ 0.54
	4	18.50 $\pm$ 0.53	22.82 $\pm$ 0.53	19.53 $\pm$ 0.44
	8	16.47 $\pm$ 0.11	24.42 $\pm$ 0.62	22.64 $\pm$ 0.20
PubMed	1	8.48 $\pm$ 0.12	8.07 $\pm$ 0.11	8.31 $\pm$ 0.15
	2	8.84 $\pm$ 0.16	9.19 $\pm$ 0.11	9.09 $\pm$ 0.19
	4	10.07 $\pm$ 0.14	9.66 $\pm$ 0.12	10.13 $\pm$ 0.20
	8	9.61 $\pm$ 0.21	9.65 $\pm$ 0.15	9.36 $\pm$ 0.30
ogbl-ddi	1	12.78 $\pm$ 0.97	13.75 $\pm$ 0.75	14.15 $\pm$ 0.23
	2	13.23 $\pm$ 0.63	13.23 $\pm$ 0.20	13.51 $\pm$ 0.09
	4	13.26 $\pm$ 0.22	13.76 $\pm$ 0.38	12.99 $\pm$ 0.14
	8	11.78 $\pm$ 0.42	13.99 $\pm$ 0.34	13.96 $\pm$ 0.49

4 Discussion and Conclusion

Our method seems to work fairly well, offering a way to prevent over-smoothing while saving compute. However, we discovered that likely due to the non-enforcing nature of the algorithm, the behaviour of the method is very sensitive to hyperparameters and small implementation choices. The main issue is not that hyperparameters affect performance, but that the effect is hard to predict and rather stochastic: e.g. choosing a significantly wrong learning rate leads to our method collapsing into either instant exiting or never-exiting strategies. Or, for instance, when training a shallow GCN, it looks convergent around 500 steps to exiting very early; however, if you train it for 10_000 more steps, it will change the behaviour to exiting at the last layer! Thus the early-exit distribution dynamics throughout training warrants closer inspection.

We noticed that the subgraph method reduces these effects significantly. By enforcing spatial consensus, it acts as a structural regulariser; in fact, the subgraph method is even trainable with a hard Gumbel estimator, instead of the soft one. Thus, we believe that the subgraph method, or similar neighbourhood-aware schemes, represent the more robust path forward for real-world applications.

Limitations. Our evaluation is limited to three HearT datasets with training budgets reduced by $10\times$ relative to the standard protocol, owing to compute constraints. We report SAS-GNN results only in the Appendix for the same reason. The FLOP savings may not translate to GPU speedups without sparse-execution support (see Section ??). Finally, we evaluate only two backbone architectures (GCN and SAS-GNN); generalisation to attention-based or heterogeneous GNNs remains untested.

We hope future work could address the lack of heavy explorations about different emergent behaviours of our methods.

References

- [1] Andrea Giuseppe Di Francesco, Maria Sofia Bucarelli, Franco Maria Nardini, Raffaele Perego, Nicola Tonellotto, and Fabrizio Silvestri. Early-exit graph neural networks. *ArXiv*, abs/2505.18088, 2025. URL <https://api.semanticscholar.org/CorpusID:278886684>.
- [2] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with alphafold. *Nature*, 596(7873):583–589, August 2021. ISSN 1476-4687. doi: 10.1038/s41586-021-03819-2. URL <https://doi.org/10.1038/s41586-021-03819-2>.
- [3] Thomas N. Kipf and Max Welling. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*, 2017.
- [4] William L. Hamilton, Zhitao Ying, and Jure Leskovec. Inductive Representation Learning on Large Graphs. In *NIPS*, pages 1024–1034, 2017.
- [5] Hoang NT and Takanori Maehara. Revisiting graph neural networks: All we have is low-pass filters, 2019. URL <https://arxiv.org/abs/1905.09550>.
- [6] T. Konstantin Rusch, Michael M. Bronstein, and Siddhartha Mishra. A survey on oversmoothing in graph neural networks, 2023. URL <https://arxiv.org/abs/2303.10993>.
- [7] Haimin Zhang, Min Xu, Guoqiang Zhang, and Kenta Niwa. Ssf: Stochastically scaling features and gradients for regularizing graph convolutional networks. *IEEE Transactions on Neural Networks and Learning Systems*, 35(2):2223–2234, February 2024. ISSN 2162-2388. doi: 10.1109/tnnls.2022.3188888. URL <http://dx.doi.org/10.1109/TNNLS.2022.3188888>.
- [8] T. Konstantin Rusch, Ben Chamberlain, James Rowbottom, Siddhartha Mishra, and Michael Bronstein. Graph-coupled oscillator networks. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 18888–18909. PMLR, 17–23 Jul 2022. URL <https://proceedings.mlr.press/v162/rusch22a.html>.
- [9] Simone Scardapane, Michele Scarpiniti, Enzo Baccarelli, and Aurelio Uncini. Why should we add early exits to neural networks? *Cognitive Computation*, 12(5):954–966, September 2020. ISSN 1866-9964. doi: 10.1007/s12559-020-09734-4. URL <https://doi.org/10.1007/s12559-020-09734-4>.
- [10] Indro Spinelli, Simone Scardapane, and Aurelio Uncini. Adaptive propagation graph convolutional network. *IEEE Transactions on Neural Networks and Learning Systems*, 32(10):4755–4760, October 2021. ISSN 2162-2388. doi: 10.1109/TNNLS.2020.3025110. URL <https://doi.org/10.1109/TNNLS.2020.3025110>.
- [11] Juanhui Li, Harry Shomer, Haitao Mao, Shenglai Zeng, Yao Ma, Neil Shah, Jiliang Tang, and Dawei Yin. Evaluating graph neural networks for link prediction: Current pitfalls and new benchmarking. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 3853–3866. Curran Associates, Inc.,

2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/Obe50b4590f1c5fdf4c8feddd63c4f67-Paper-Datasets_and_Benchmarks.pdf.

- [12] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with gumbel-softmax, 2017. URL <https://arxiv.org/abs/1611.01144>.

A SAS exit plot

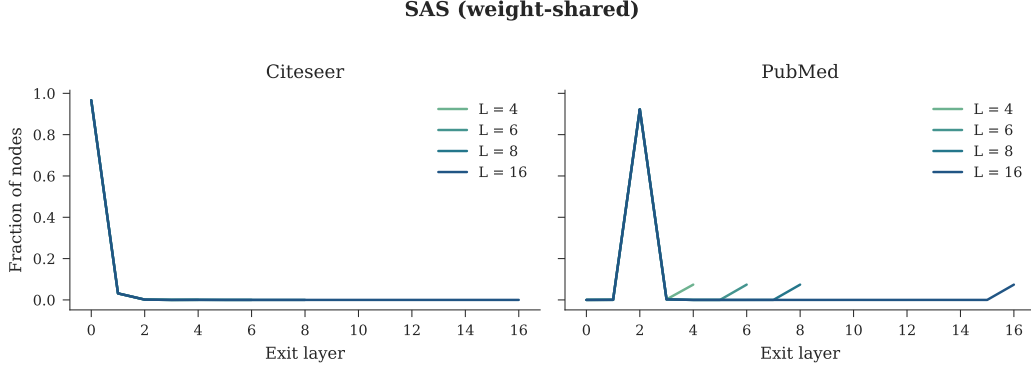


Figure 5: Early exiting depth distribution of SAS for PubMed and Citeseer. SAS seems to be a lot more prone to undesirable collapse than GCN, but also a lot more consistent.

B SAS results plot

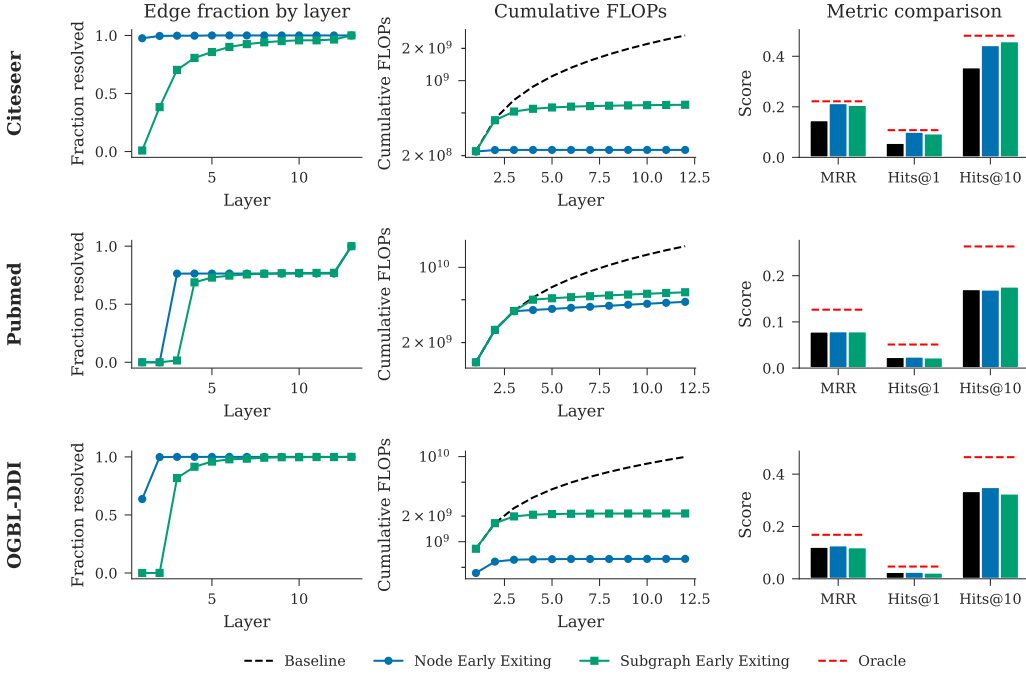


Figure 6: Evaluation of SAS via HeaRT on a few datasets. Our method outperforms the baseline, but does not reach the oracle performance.